

Name: Peram.Mahitha

Reg no: 192524136

1. Scenario: Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.

Task:

A. Input: A grayscale image (size $M \times N$)

B. Output: Bounding boxes of detected faces and corresponding recognized names

C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode Task:

1. Preprocess the image (grayscale normalization, histogram equalization)

2. Detect faces using a sliding window or Viola-Jones detector

3. Extract features using Eigenfaces

4. Compare with database of known faces using Euclidean distance

5. Output bounding boxes with recognized labels

A) Pseudo code:

ALGORITHM Face _ Detection _ Recognition _ Eigenfaces

INPUT:

Grayscale image I of size $M \times N$

Database of known face images with names

OUTPUT:

Bounding boxes of detected faces

Recognized name for each detected face

BEGIN

1. LOAD the input grayscale image I.

2. PREPROCESS IMAGE:

a. Normalize the grayscale intensity values of I.

b. Apply Histogram Equalization to improve contrast and reduce the effect of varying lighting conditions.

3. FACE DETECTION:

a. Apply Viola-Jones face detector OR sliding window technique on the preprocessed image.

b. Detect all faces present in the image.

c. For each detected face, store its bounding box coordinates (x, y, width, height).

4. FOR each detected face DO

a. CROP the face region using its bounding box.

b. RESIZE the cropped face to a fixed size.

c. NORMALIZE the resized face.

d. CONVERT the face image into a vector.

e. PROJECT the face vector onto the Eigenfaces space to obtain its Eigenface features.

f. FOR each known face in the database DO

i. Obtain its Eigenface feature vector.

ii. Calculate Euclidean distance between the detected face features and database features.

END FOR

g. FIND the database face having the minimum Euclidean distance.

h. IF the minimum distance is less than the

recognition threshold THEN

Assign the corresponding person's name.

ELSE

Assign label "Unknown".

END IF

i. STORE the bounding box and recognized name.

END FOR

5. DISPLAY the original image.

6. DRAW bounding boxes around all detected faces.

7. DISPLAY the recognized name above each bounding box.

8. RETURN all bounding boxes and corresponding names.

END

2. Scenario: Design an algorithm to detect road lanes in a real-time driving video sequence.

Task:

A. Input: Video frames from a front-facing camera

B. Output: Lane lines highlighted on each frame

C. Requirements: Robust to shadows, curves, and partial occlusion

Pseudocode Task:

1. Capture video frame

2. Convert frame to grayscale

3. Apply Gaussian smoothing to reduce noise

4. Perform edge detection (Canny)

5. Apply Hough Transform to detect straight/curved lines

6. Overlay detected lanes on the original frame

7. Repeat for all frames

A) Pseudo code:

ALGORITHM Road _ Lane _ Detection

INPUT:

Video sequence from a front-facing camera

OUTPUT:

Video frames with detected lane lines highlighted

BEGIN

1. START the video stream from the front-facing camera.
2. WHILE video frames are available DO
 - a. CAPTURE the current video frame.
 - b. CONVERT the frame from RGB to grayscale.
 - c. APPLY Gaussian Smoothing to the grayscale image to reduce noise and small unwanted details.
 - d. APPLY Canny Edge Detection to identify strong edges in the road image.
 - e. SELECT the Region of Interest (ROI) containing the road area.
 - f. APPLY Hough Transform to the detected edge to identify possible lane lines.
 - g. GROUP and FILTER the detected lines to remove irrelevant lines such as vehicle edges or road objects.
 - h. ESTIMATE the left and right lane boundaries.

For curved roads, use a suitable curve-fitting method such as polynomial fitting.
 - i. HANDLE partial occlusion by using lane information from previous frames to estimate missing lane portions.
 - j. DRAW the detected left and right lane line on the original video frame.
 - k. DISPLAY the processed frame with highlighted lane lines.

3. END WHILE

4. STOP the video stream.

END